



线程

后续关注公众号[研大]
永久微信1931657888

基本概念

状态&转换

线程管理的数据结构

控制

实现方式





线程

基本概念

状态&转换

线程管理的数据结构

控制

实现方式

一个进程只有一个执行点

(一个程序计数器，用来存放要执行的指令)

但多线程(multi-threaded)进程会有多个执行点

(多个逻辑上的程序计数器，每个都用于取指令和执行)

每个线程类似于独立的进程，只有一点区别：它们共享进程的地址空间，从而能够访问相同的数据



单线程的地址空间

后续关注公众号「研木」
永久微信1951657115



多线程的地址空间





线程

基本概念

状态&转换

线程管理的数据结构

控制

实现方式

一个进程只有一个执行点

(一个程序计数器，用来存放要执行的指令)

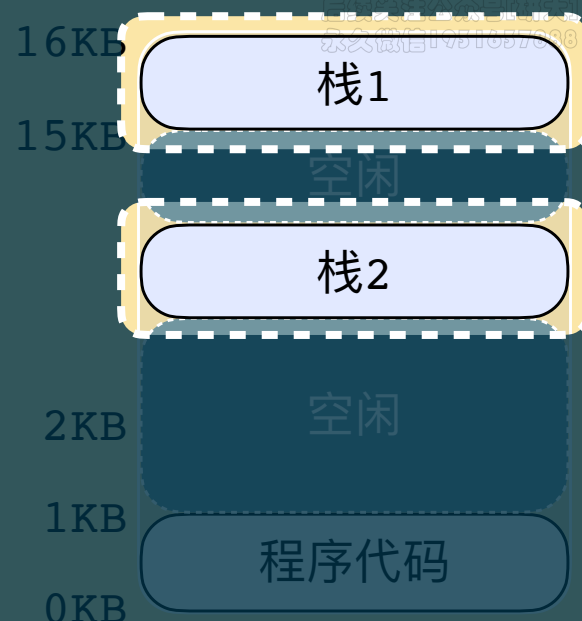
但多线程(multi-threaded)进程会有多个执行点

(多个逻辑上的程序计数器，每个都用于取指令和执行)

每个线程类似于独立的进程，只有一点区别：它们共享进程的地址空间，从而能够访问相同的数据



单线程的地址空间



多线程的地址空间

每个线程都有自己独立的栈，并且线程之间没有做保护措施，每个线程的栈可以被其他线程读写甚至清除，由一个线程打开的文件也可以被其他线程读写

线程只是作为独立调度和分派的单位，但是它们没有独立的地址空间，也没有掌握资源，完全依附于进程





线程

基本概念

状态

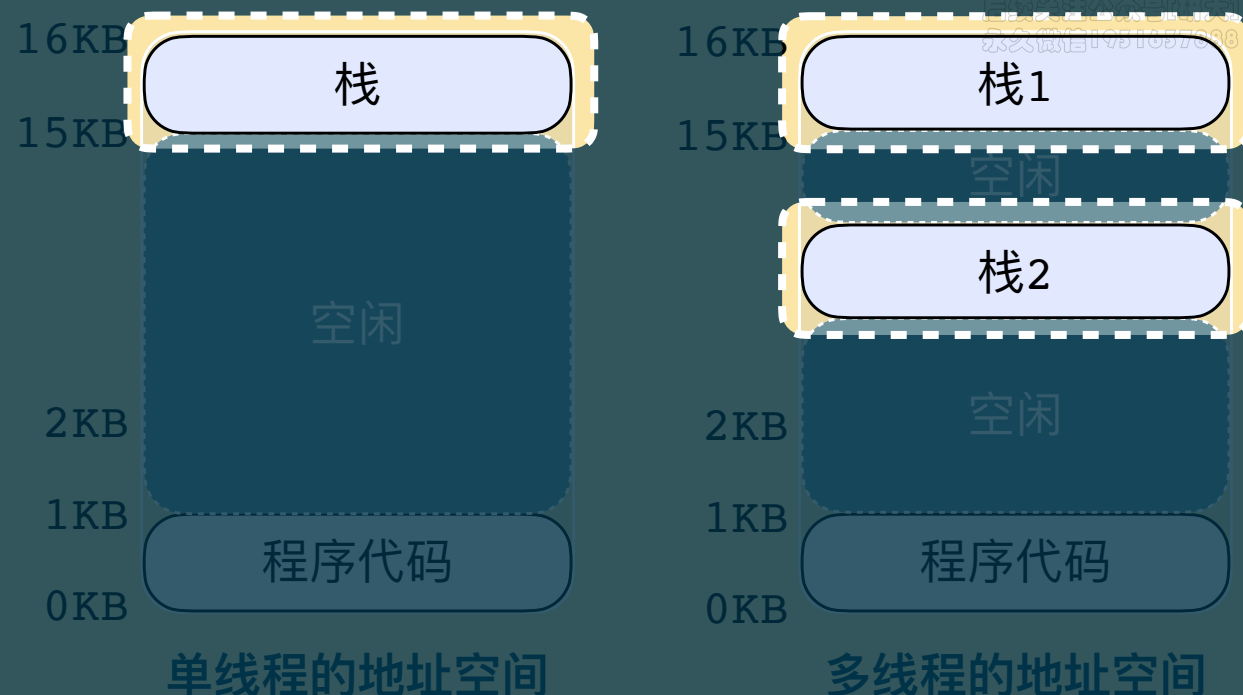
一个进程只有一个执行点
(一个程序计数器，用来存放要执行的指令)

但多线程(multi-threaded)进程会有
(多个逻辑上的程序计数器，每个都用于取指令和执

每个线程类似于独立的进程，只有一点区别：它们共享进程的地址空间

每个线程都有自己独立的栈，并且线程之间没有做保护措施，每个线程的栈可以被其他线程读写甚至清除，由一个线程打开的文件也可以被其他线程读写

线程只是作为独立调度和分派的单位，但是它们没有独立的地址空间，也没有掌握资源，完全依附于进程



每个进程中内容	每个线程中内容
地址空间	程序计数器
全局变量	寄存器
打开文件	栈
子进程	状态
堆	...
...	





线程

基本概念

状态&转换

线程管理的数据结构

控制

实现方式

就绪态

执行态

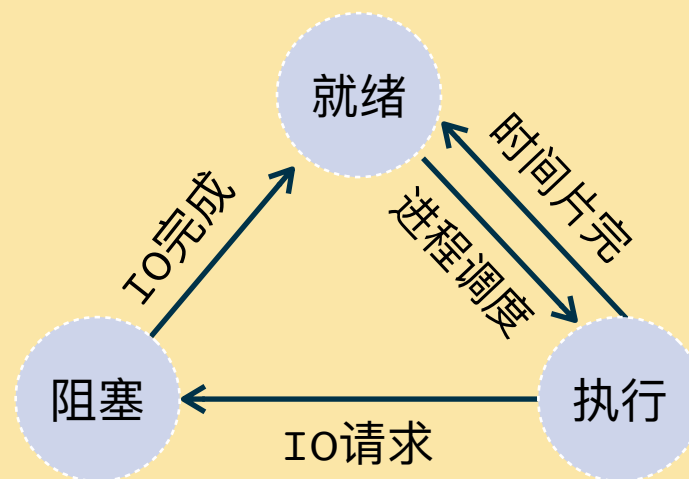
阻塞态

后续关注公众号「研大」
永久微信1951657888

线程已具备执行条件，只差CPU

线程获得CPU并且正在运行

线程在执行中因某件事而受阻
处于暂停状态





线程

[基本概念](#)[状态&转换](#)[线程管理的数据结构](#)[控制](#)[实现方式](#)

TCB中的信息

线程标识符

为每个线程赋予一个唯一的线程标识符

一组寄存器的内容

PC、PSW、通用寄存器等

线程执行状态

描述线程当前正处于何种状态

优先级

描述线程执行的优先程度

线程专有存储区

用于在线程切换时存放现场保护信息和
与该线程相关信息

信号屏蔽

对某些信号加以屏蔽

堆栈指针

必须设置指向用户自己堆栈的指针和指
向内核栈的指针





线程

[基本概念](#)[状态&转换](#)[线程管理的数据结构](#)[控制](#)[实现方式](#)

线程的创建

用户程序在启动时，通常只有一个初始化线程在执行，它利用一个线程创建函数来创建新线程，并返回一个线程标识符

后续关注公众号[研大]
永久微信1951657888

线程的终止

终止线程通过调用相应的函数执行终止操作，大多数os中，线程被终止后不会立即释放它所占有的所有资源，只有当进程中的其他线程执行了**分离函数**后，被终止的线程才会与资源分离，此时的资源才能被其他线程利用

已被终止但尚未释放资源的线程，仍可被需要它的线程所调用以恢复运行



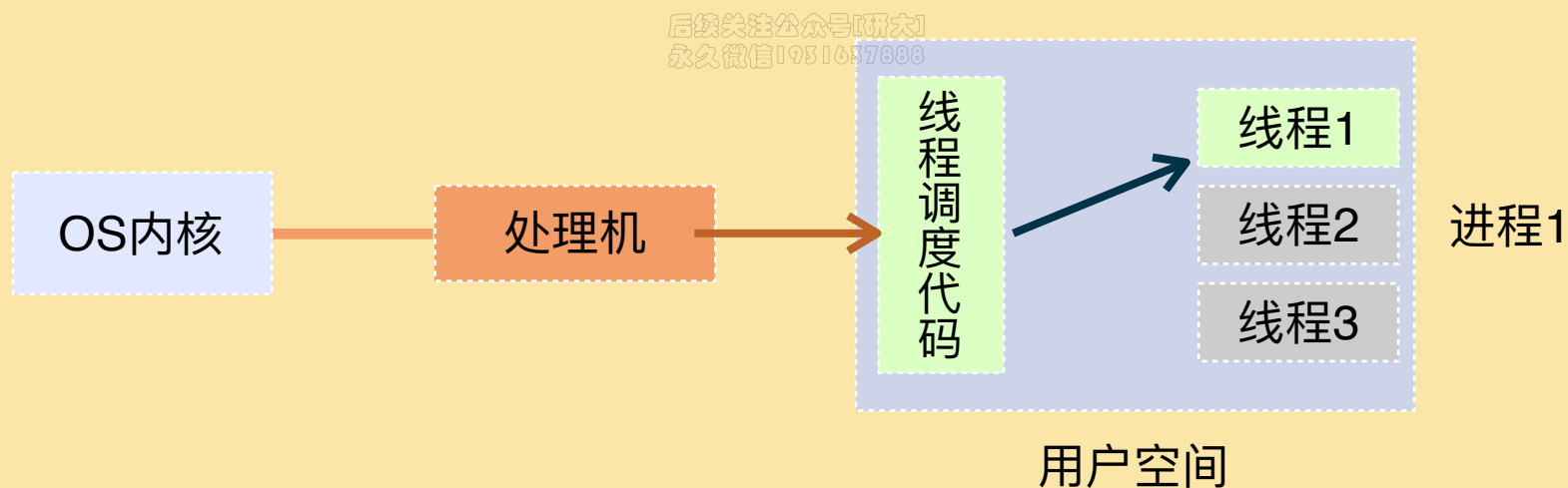


线程

[基本概念](#)[状态&转换](#)[线程管理的数据结构](#)[控制](#)[实现方式](#)

用户级线程ULT

把整个线程实现部分放在用户空间中，内核看到的就是一个单线程进程





基本概念

状态&转换

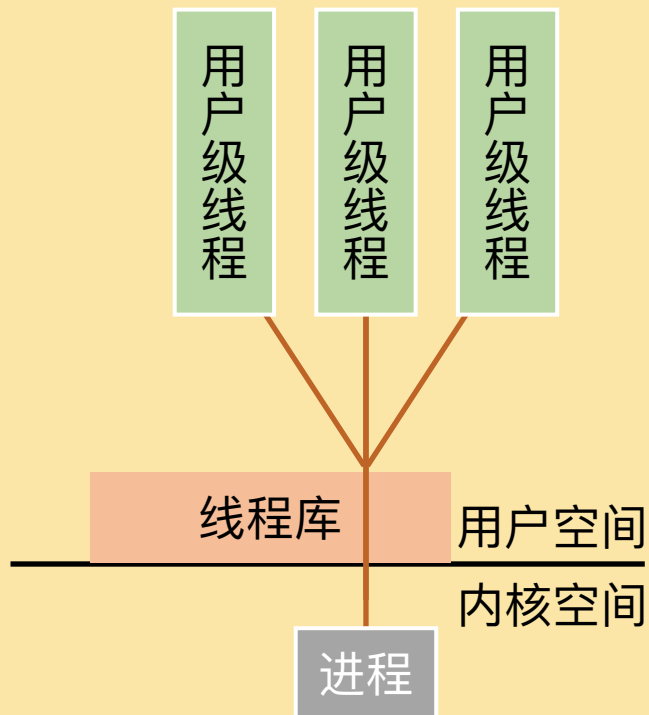
线程管理的数据结构

控制

实现方式

用户级线程ULT

把整个线程实现部分放在用户空间中，内核看到的就是一个单线程进程



1. 线程切换在用户空间实现，无需模式切换
2. 每个进程可以选择自己专用的调度算法，对自己内部的线程进行调度和管理
3. ULT的实现与os无关，因为ULT管理的代码会显式地出现在用户程序中，因此ULT甚至可以在不支持线程机制的os平台上实现

1. 内核只识别到一个进程，因此调度的对象是整个用户进程，每次只能给整个进程分配CPU，进程中只有一个线程能执行，无法发挥多处理器系统优势
2. 进程中的某个线程发起IO系统调用，会导致进程中的所有线程全部被阻塞





基本概念

状态&转换

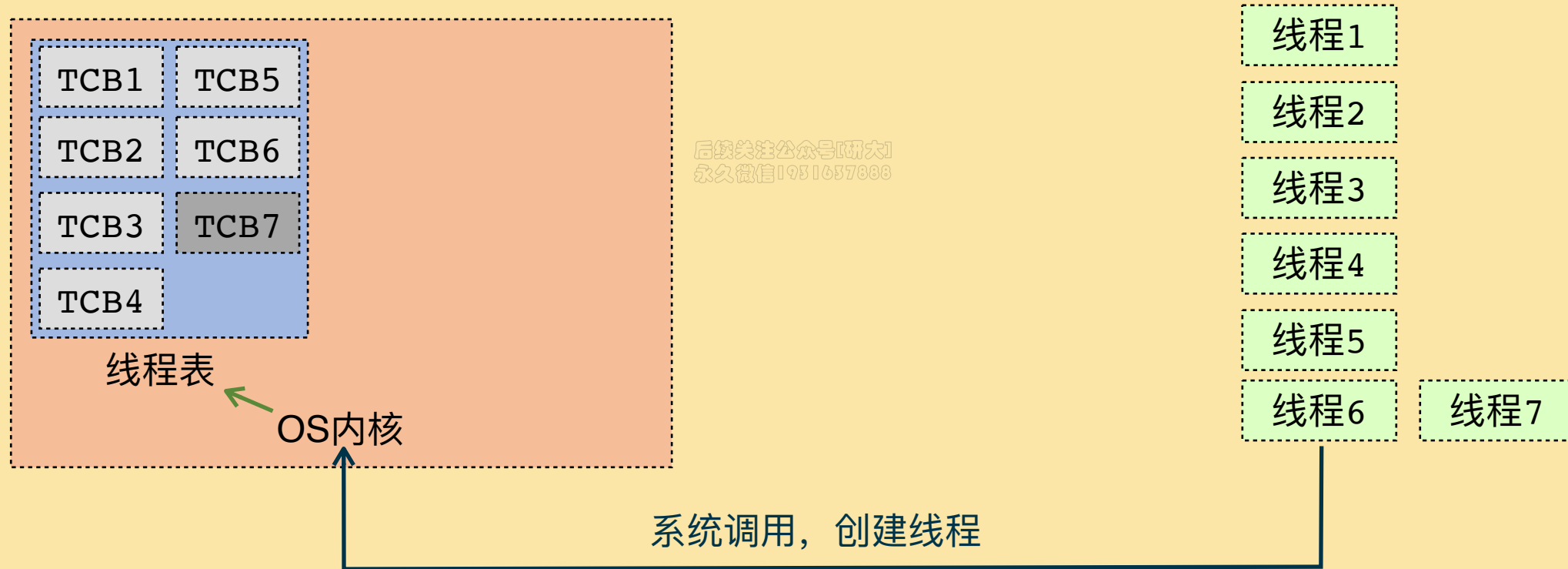
线程管理的数据结构

控制

实现方式

内核支持线程KST

在内核的支持下运行的线程，是操作系统可感知的线程





基本概念

状态&转换

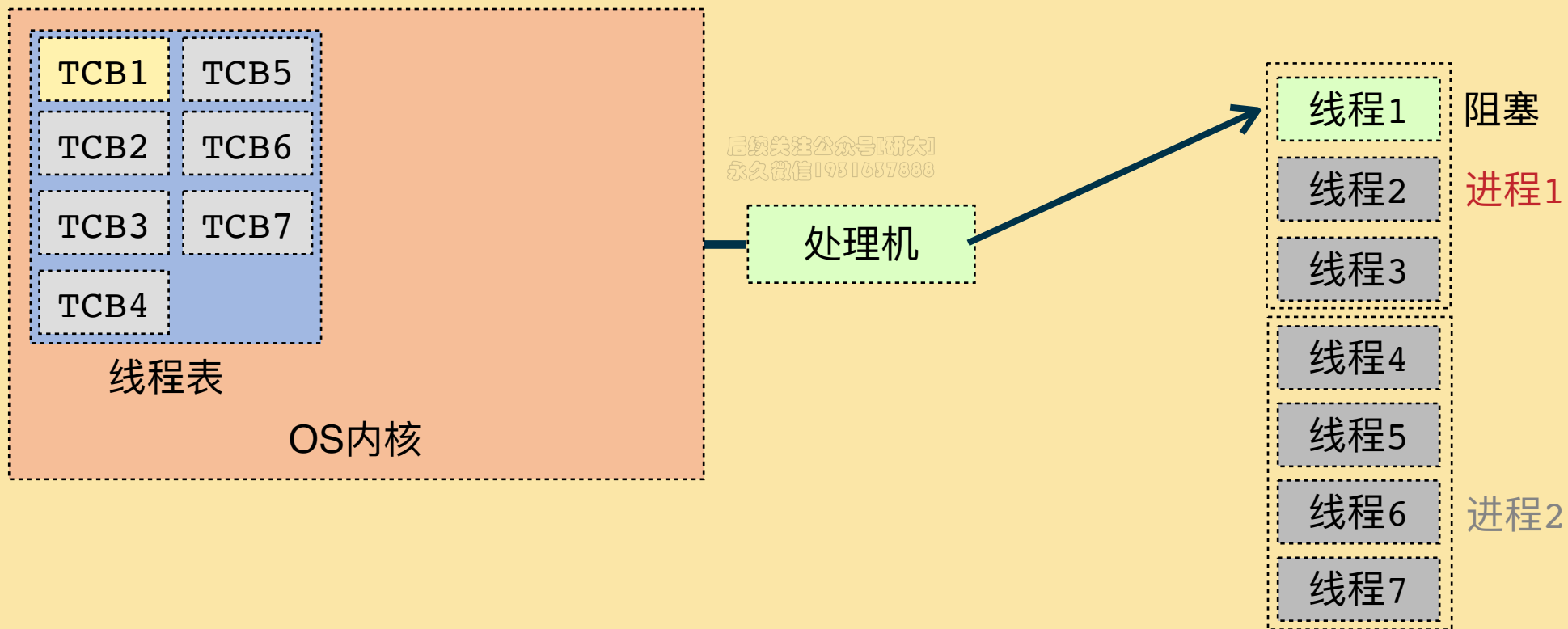
线程管理的数据结构

控制

实现方式

内核支持线程KST

在内核的支持下运行的线程，是操作系统可感知的线程





基本概念

状态&转换

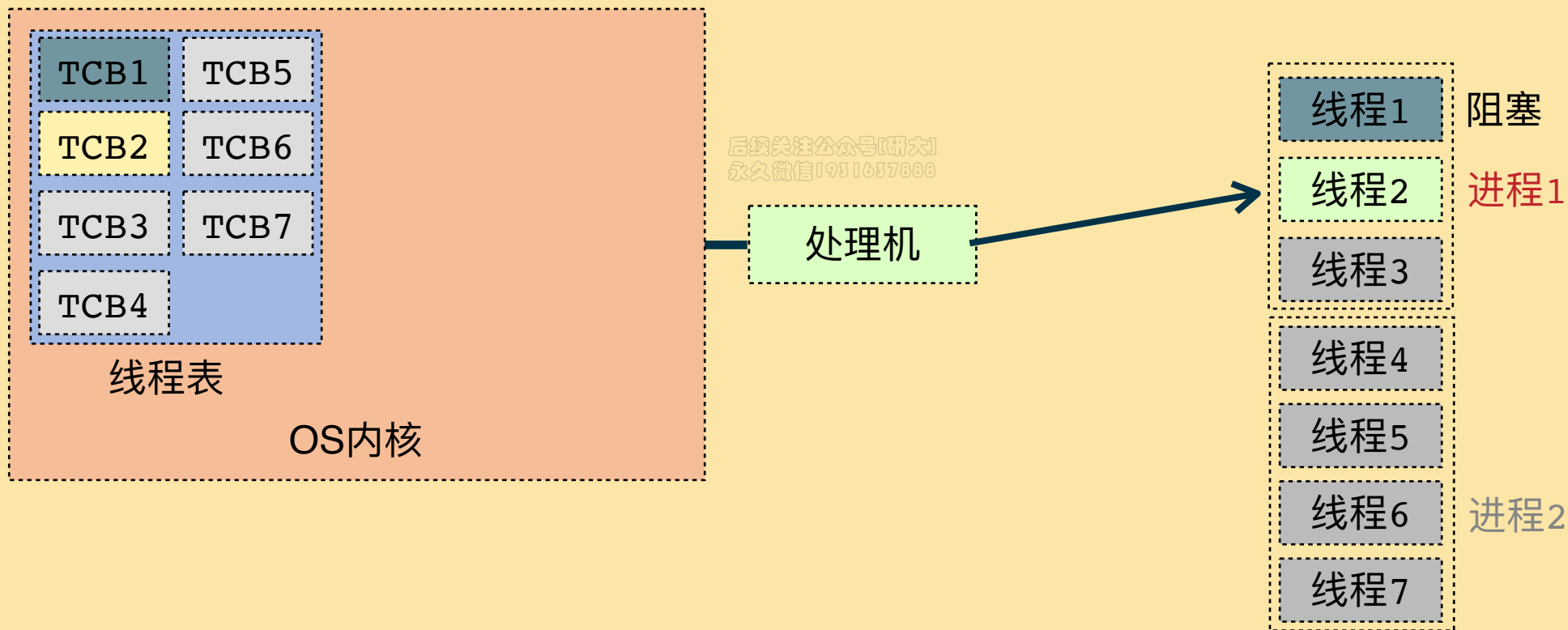
线程管理的数据结构

控制

实现方式

内核支持线程KST

在内核的支持下运行的线程，是操作系统可感知的线程





基本概念

状态&转换

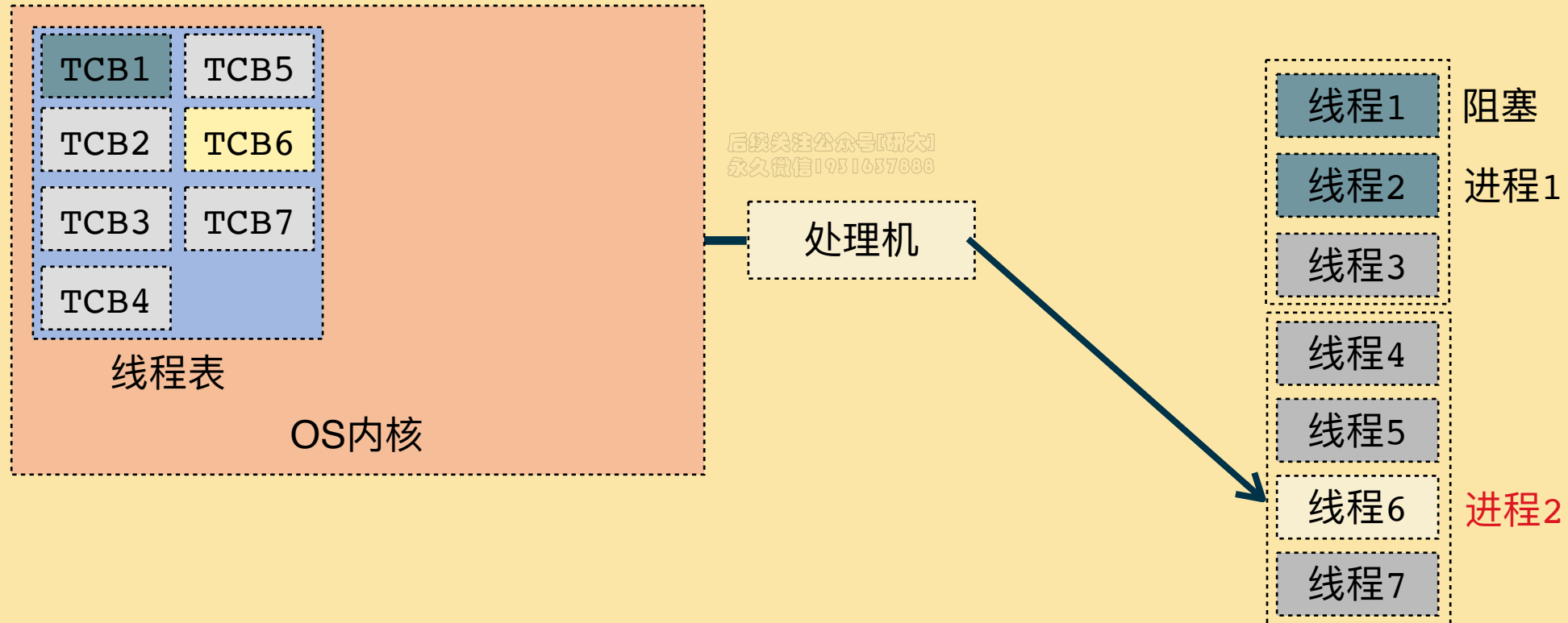
线程管理的数据结构

控制

实现方式

内核支持线程KST

在内核的支持下运行的线程，是操作系统可感知的线程





基本概念

状态&转换

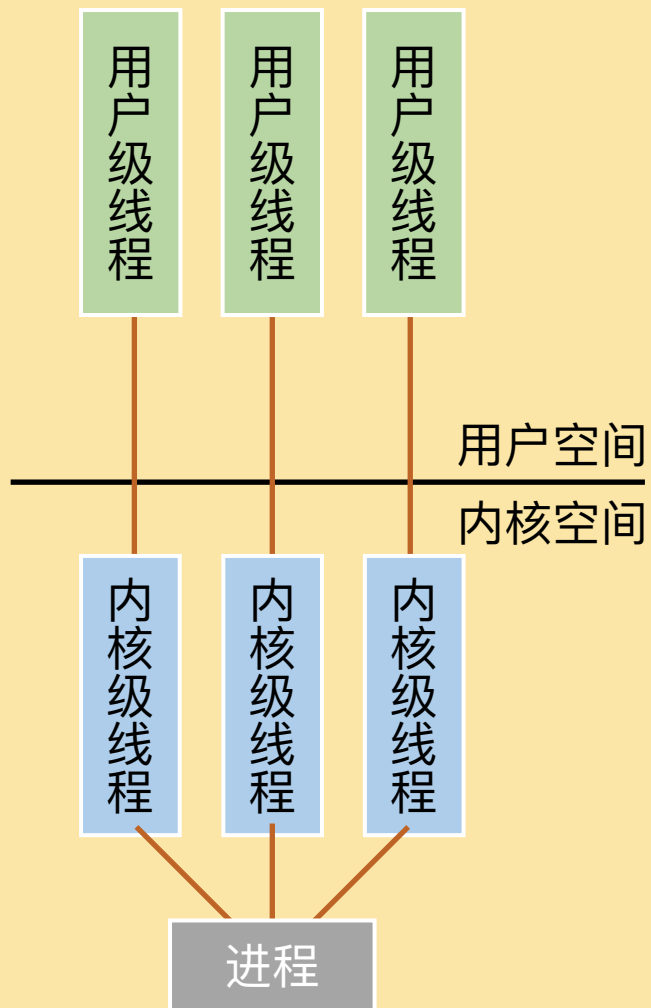
线程管理的数据结构

控制

实现方式

内核支持线程KST

在内核的支持下运行的线程，是操作系统可感知到的线程



1. 内核可识别到内核级线程，因此调度的对象是内核级线程，在多台处理机系统中，每次可以给不同的内核级线程分配CPU，提高并行度
2. 如果用户进程中的一个线程被阻塞，内核可以调度该进程内的其他线程上处理机，或者调度其他进程的线程

1. 每次线程切换都需要CPU变态，开销较大
2. TCB存放在内核空间中，内存占用较大





线程

基本概念

状态&转换

线程管理的数据结构

控制

实现方式

两种实现方式的比较

后续关注公众号[研大]
永久微信1951457000

	用户级线程ULT	内核支持线程KST
调度	开发者自己实现 内核无法感知	内核来实现 内核可以感知
开销	线程管理开销远小于KST	开销较大

线程只能在进程内部进行资源竞争 线程可以在全系统内进行资源的竞争



线程



基本概念

状态&转换

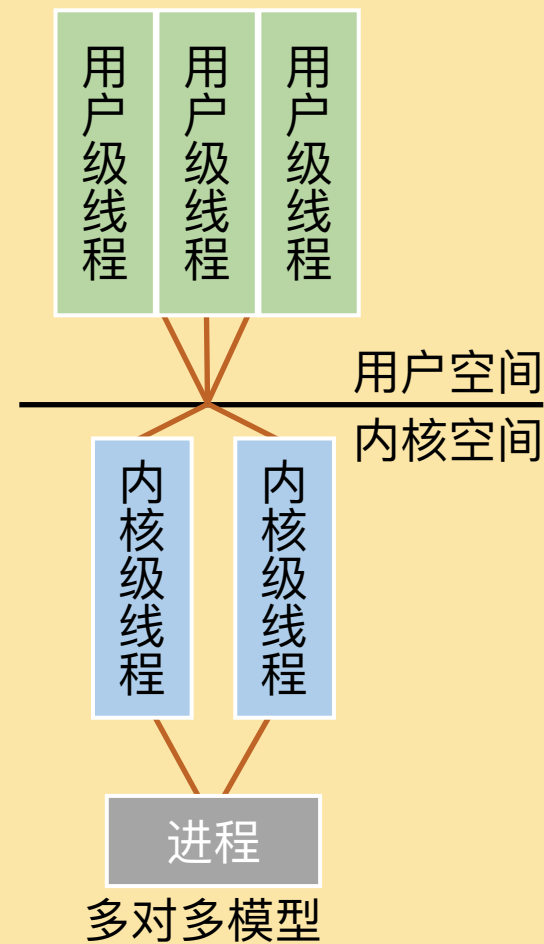
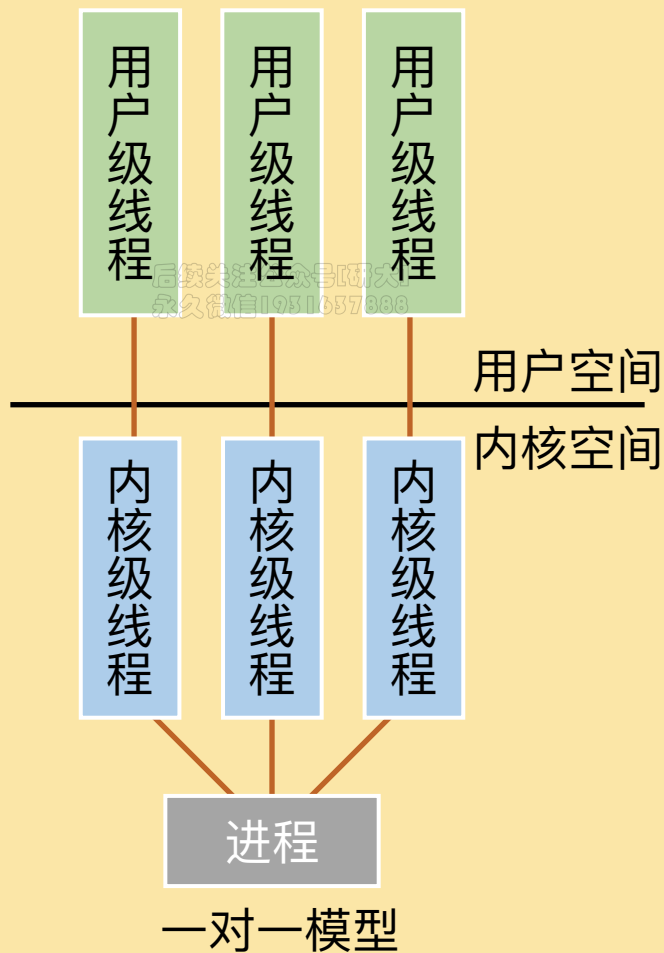
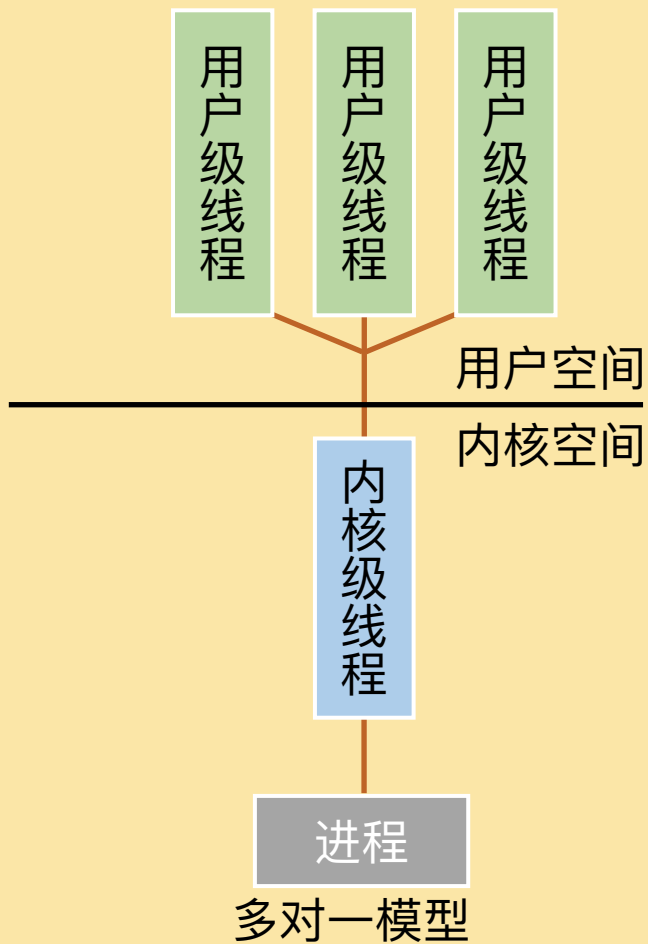
线程管理的数据结构

控制

实现方式

同时支持ULT和KST的OS

线程的实现





基本概念

状态&转换

线程管理的数据结构

控制

实现方式

同时支持ULT和KST的OS

线程的实现

多对一模型

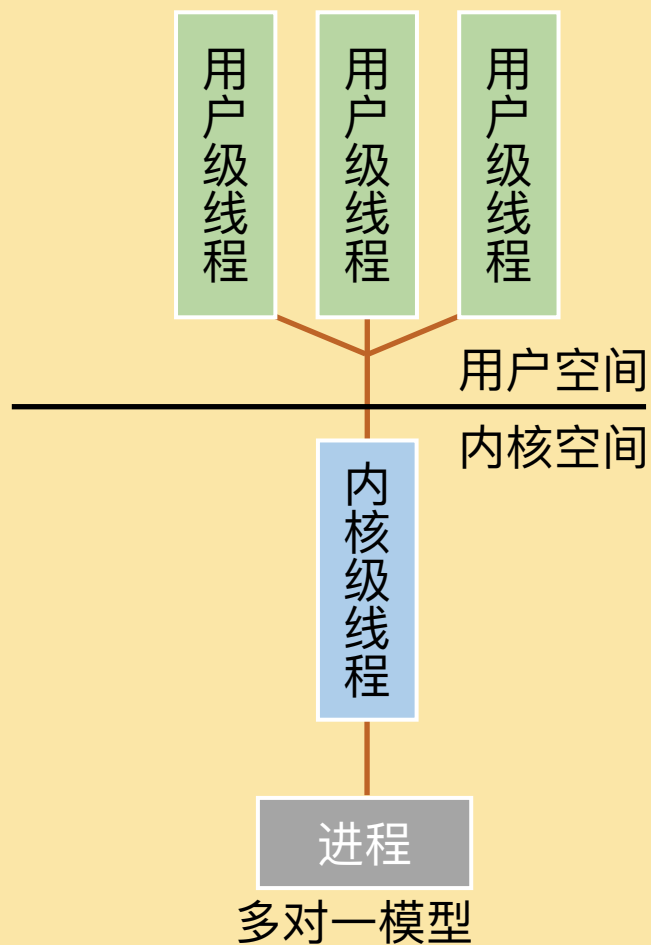
把多个用户级线程映射到一个内核级线程

线程管理在用户空间完成

用户级线程对操作系统不可见

优点：线程管理在用户空间进行，效率高

缺点：一个线程在使用内核服务时被阻塞，整个进程都会被阻塞





基本概念

状态&转换

线程管理的数据结构

控制

实现方式

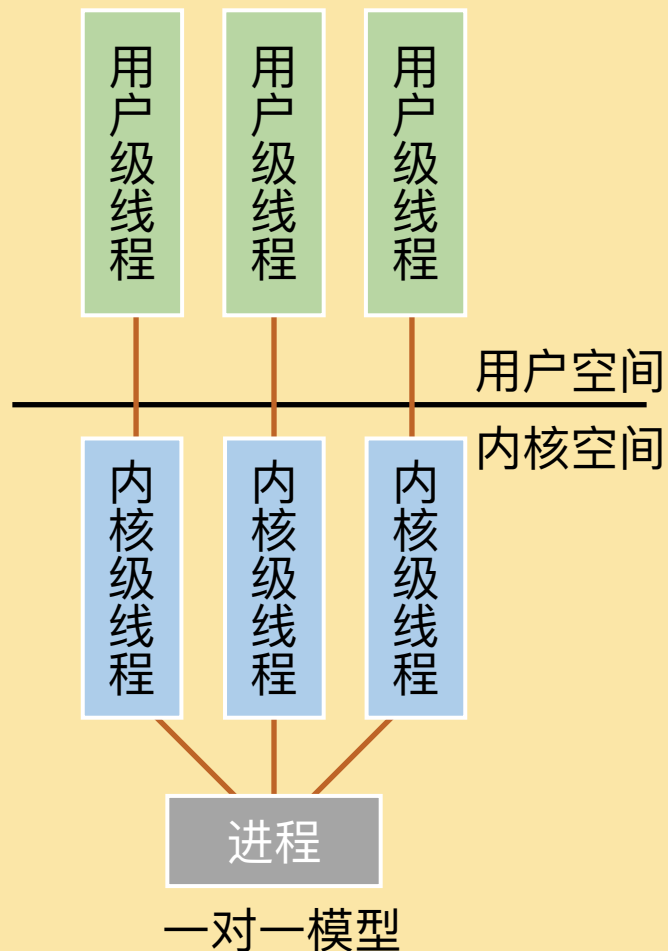
同时支持ULT和KST的OS

线程的实现

一对一模型

每个用户级线程都映射到一个内核级线程

后续关注微信公众号
永久微信1951657888



优点： 当一个线程被阻塞时，允许另一个线程继续运行，并发能力强

缺点： 每个用户级线程都需要一个内核级线程与其对应，线程管理开销很大





基本概念

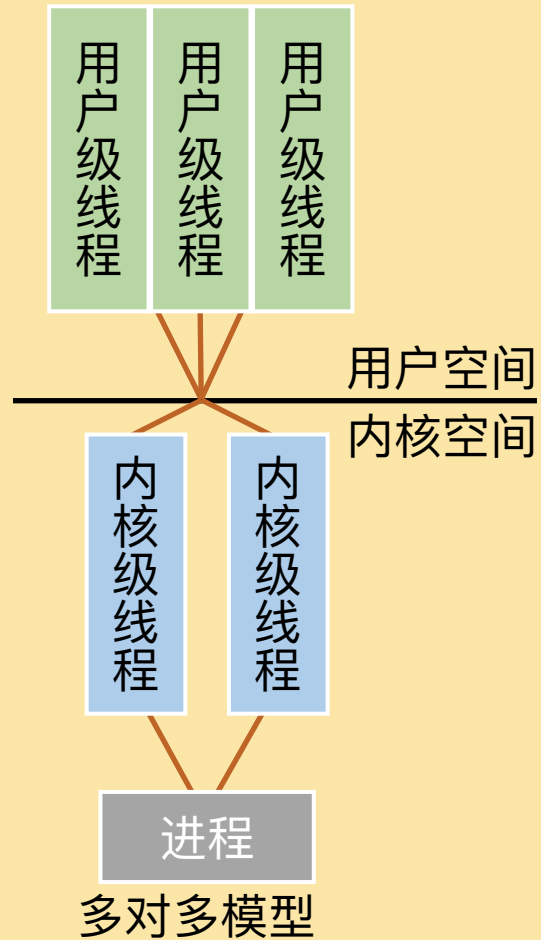
状态&转换

线程管理的数据结构

控制

实现方式

同时支持ULT和KST的OS
线程的实现



后续关注公众号「研大」
永久微信1951657888

多对多模型

n 个用户级线程映射到 m 个内核级线程上, $m \leq n$

结合一对一模型和多对一模型的灵活使用

